Create a clean v1.5.48 production release file.

1.2
Recommendation: in v1.5.48, Section F should validate number/date format only when the dashboard column definition has dateColumn === true. Non-date text columns should not trigger default number-format failures.

Recommendation: change the governed default behavior for non-date template data cells to text ("@") or, better, do not validate non-date number format in Section F unless a column explicitly declares a special number format.

1.4

Recommendation: for v1.5.48, Raw Data needs a governed template-application function that applies the Template - Raw Data formatting to the existing active Raw Data output sheet without leaving source/duplicate Raw Data sheets behind.

1.5

It is dashboard-driven, but it is still multi-step and range-operation-heavy. The spec's One Pass Processing concept is more naturally aligned to data/business processing, but the template/formatting layer should still be optimized toward a "single resolved formatting plan" and minimal batched writes.

2. Performance Bottlenecks
2.1
Recommendation: in v1.5.48, skip row-resize logic entirely when rowCount <= 1000. Google Sheets already creates new sheets with 1000 rows, so those templates should not pay any row-resize cost.

2.2
For Monthly Change, verify subsection title row, spacer row, subsection header row, and formatting block.

Validate date format only for dashboard columns where dateColumn === true.

If no issues exist, write one PASS summary row.

2.3 - the formatting is linked to the column headers and should be the same across all templates.
Recommendation: for v1.5.48, build a resolved template-formatting plan in memory first, then apply it in larger, predictable blocks. Avoid repeated column-by-column validation and formatting where possible.

3. Architecture Issues
3.1
The current checked-in script is still a previous release artifact and should not be patched again.

3.2 create a separate governed formatting stage:--- What would this do? With the Raw Data why can't it's Formatting use the same path as the other sheets. Create copy of template, copy and paste data to sheet.

Recommendation: create a separate governed formatting stage:

3.3
Recommendation: split validation responsibilities:

Section F: template existence, headers, required rows, Monthly Change subsections, date-format columns only.

Formatting diagnostics / optional QA: widths, hidden columns, default text format, row heights, filters.

Performance report: separate timing by first build, full rebuild, metadata-only refresh.

3.4 Monthly Change subheader structure still needs final output validation
The script has a Monthly Change subheader guide function and a spacer formatting function.

However, Section F currently does not show a complete dedicated validation sequence for:

subsection title row;

spacer/dark row;

subsection header row;

copied headers in 8pt font;

formatting block.

The user's visual requirement is specific: after the dark/spacer row, the next row should copy the headers in 8pt font. That needs to be validated by Section F and verified in final output.

Analysis Only — v1.5.47.1 Review / v1.5.48 Direction

I checked out work from origin/main and reviewed the checked-in script plus the available report files. The exact (1) filenames with spaces before (1) were not present in Report/; the available report set is:

Report/v1.5.47.1 - Dashboard Quality Report.pdf

Report/v1.5.47.1 - Framework Timing Report.pdf

Report/v1.5.47.1 - Google Apps Scripts Executions.pdf

Commands used for this analysis:

git checkout -B work origin/main

find Report -maxdepth 1 -type f | sort

find Current_Production -maxdepth 1 -type f | sort

python3 -m pip install --quiet --target /tmp/ml_py pypdf openpyxl

PYTHONPATH=/tmp/ml_py python3 ... to extract/report-review the PDFs

rg -n ... Current_Production/v1.5.47.1_Production_Script.txt

nl -ba Current_Production/v1.5.47.1_Production_Script.txt | sed ...

git status --short

No files were modified.

1. Critical Issues
1.1 v1.5.47.1 is still internally versioned as v1.5.47
The checked-in Current_Production/v1.5.47.1_Production_Script.txt still declares:

const MASTER_LIST_MERGE_ML_VERSION = "1.5.47";
That means reports generated from this script will continue to title themselves as v1.5.47, even if the filename is v1.5.47.1. This is consistent with the report titles extracted from the PDFs. The version constant must not be patched again in v1.5.47.1; the next production script should be a full release file named Current_Production/v1.5.48_Production_Script.txt with the internal version set to 1.5.48.

Recommendation: do not keep editing v1.5.47.1. Create a clean v1.5.48 production release file.

1.2 Section F is still failing because default number format validation is active

The Dashboard Quality report shows Section F failures dominated by messages like:

Date/number format mismatch ... expected General but found 0.################

The script still defines the default number format as "General" and still uses that default for non-date columns.

Section F then validates every expected header's number format, regardless of whether the column is actually a date column. That is why non-date fields such as names, phone numbers, address fields, language fields, notes, and Primary PMR Row are being checked against "General".

This conflicts with the current requirement:

Section F should only check columns that are true for Date Format, and there should be number/date format validation only where governed by the dashboard date-format requirement.

Recommendation: in v1.5.48, Section F should validate number/date format only when the dashboard column definition has dateColumn === true. Non-date text columns should not trigger default number-format failures.

1.3 Default text format exists but is not the active default for non-date columns
The script already contains defaultTextFormat: "@", but getExpectedNumberFormat_() does not use it for ordinary non-date fields. Instead, non-date fields fall back to defaultNumberFormat, which is still "General".

That means the desired "default format by text, not General number" is not implemented in the validation path.

Recommendation: change the governed default behavior for non-date template data cells to text ("@") or, better, do not validate non-date number format in Section F unless a column explicitly declares a special number format.

1.4 Raw Data still does not receive real template formatting
formatRawData() calls applyDashboardTemplateFormattingToActiveReportSheet_().

However, that function explicitly says runtime report formatting is restricted and may not reapply dashboard formatting, hidden columns, filters, freezes, widths, or date/number formatting outside the template builder.

The actual implementation only writes metadata/date cells, applies final row-height lock, clears caches, and returns. It does not copy full template styling onto the active Raw Data sheet.

This explains the user-observed issue:

Raw Data has no template formatting.

Recommendation: for v1.5.48, Raw Data needs a governed template-application function that applies the Template - Raw Data formatting to the existing active Raw Data output sheet without leaving source/duplicate Raw Data sheets behind.

1.5 The current template/formatting path is not true One Pass Processing
For templates, the current builder performs many separate formatting operations:

base sheet range formatting

title rows

header row

data row base style

column widths

date/number formats

hidden column settings

filter setup

Monthly Change subheader guide

row heights

These are separate range/API operations, not a single consolidated formatting pass.

So, to answer your question directly:

No — the current templates/formatting implementation is not using true One Pass Processing.

It is dashboard-driven, but it is still multi-step and range-operation-heavy. The spec's One Pass Processing concept is more naturally aligned to data/business processing, but the template/formatting layer should still be optimized toward a "single resolved formatting plan" and minimal batched writes.

2. Performance Bottlenecks
2.1 Create / Refresh All Templates is still over benchmark
The Framework Timing report shows:

Create / Refresh All Templates: 268.269 seconds

Benchmark: 240 seconds

Status: CRITICAL

Variance: +28.269 seconds

The report identifies row resizing as a major slow path:

Resize rows: Template - Master List (500 rows) — 11.822 sec

Resize rows: Template - Monthly Change (1000 rows) — 13.666 sec

Resize rows: Template - Disenrolled Exclusion (1000 rows) — 14.562 sec

Resize rows: Template - Master List Change Log (1000 rows) — 15.839 sec

The script still calls resizeSheetRows_(sheet, rowCount) unconditionally during full template build and then records a timing step for it.

The helper avoids deleting extra rows, but it still calls getMaxRows() and still runs for templates with 1000 rows or less.

Recommendation: in v1.5.48, skip row-resize logic entirely when rowCount <= 1000. Google Sheets already creates new sheets with 1000 rows, so those templates should not pay any row-resize cost.

2.2 Dashboard Quality Validate Templates is still far over benchmark
The Framework Timing report shows:

Dashboard Quality Validate Templates: 159.974 seconds

Benchmark: 30 seconds

Status: CRITICAL

Variance: +129.974 seconds

The major contributor is:

Dashboard Quality Section F staged: 126.532 seconds

This aligns with the code: Section F is validating not just required structure, but also freeze settings, column widths, date/number formats across sampled data rows, filters, and template signatures.

Recommendation: reduce Section F to the governed requirements only:

Load dashboard configuration.

For each governed sheet definition, locate template.

Verify template exists.

Verify template title rows exist where required.

Verify template header row exists.

Compare template headers to dashboard header definitions.

Identify missing template headers.

Identify unexpected template headers.

Validate required template sections/subsections.

For Monthly Change, verify subsection title row, spacer row, subsection header row, and formatting block.

Validate date format only for dashboard columns where dateColumn === true.

If no issues exist, write one PASS summary row.

2.3 Formatting operations are not consolidated enough
The template builder currently uses many individual operations: full-range style, title rows, header row, data rows, widths, number formats, hidden columns, filters, subheaders, row heights.

This is not efficient enough for the number of templates and columns in the framework.

Recommendation: for v1.5.48, build a resolved template-formatting plan in memory first, then apply it in larger, predictable blocks. Avoid repeated column-by-column validation and formatting where possible.

3. Architecture Issues
3.1 Production releases are being treated like patch files

The user correctly called this out. The governing approach should be:

no patch snippets for production releases;

no continued edits to v1.5.47.1;

next script must be a full production artifact: Current_Production/v1.5.48_Production_Script.txt;

the internal version constant must match the release filename.

The current checked-in script is still a previous release artifact and should not be patched again.

3.2 Formatting authority is split between template-only rules and runtime output needs
The script says templates are the formatting authority and runtime report formatting is restricted.

That rule is good for avoiding ad-hoc formatting during business workflows, but Raw Data still needs a governed way to apply template formatting to the final output sheet. Right now the runtime function is effectively a metadata-only function, so the output sheet is not formatted.

Recommendation: create a separate governed formatting stage:

template creation owns template formatting;

output formatting applies an existing template to the final output sheet;

business processing should not contain scattered formatting decisions.

3.3 Section F is doing too much and mixing QA domains
Section F should be a template structure and required-format validation section. It should not be a broad formatting audit for every default width, every default number format, every freeze setting, and every non-date field. The current validator still checks several non-required domains.

Recommendation: split validation responsibilities:

Section F: template existence, headers, required rows, Monthly Change subsections, date-format columns only.

Formatting diagnostics / optional QA: widths, hidden columns, default text format, row heights, filters.

Performance report: separate timing by first build, full rebuild, metadata-only refresh.

3.4 Monthly Change subheader structure still needs final output validation

The script has a Monthly Change subheader guide function and a spacer formatting function.

However, Section F currently does not show a complete dedicated validation sequence for:

subsection title row;

spacer/dark row;

subsection header row;

copied headers in 8pt font;

formatting block.

The user's visual requirement is specific: after the dark/spacer row, the next row should copy the headers in 8pt font. That needs to be validated by Section F and verified in final output.

4. Prioritized Remediation Plan
Priority 1 — Create v1.5.48 as a full production release, not a patch
Do this first.

Create Current_Production/v1.5.48_Production_Script.txt.

Set MASTER_LIST_MERGE_ML_VERSION = "1.5.48".

Do not keep modifying v1.5.47.1.

Do not deliver patch snippets as the production artifact.

This directly resolves the versioning and governance issue.

Priority 2 — Fix Section F validation scope
Section F should be rewritten or heavily simplified to match the stated requirements:

template exists;

title rows exist where required;

header row exists;

expected headers match dashboard definitions;

missing headers identified;

unexpected headers identified;

required subsections validated;

Monthly Change subsection title/spacer/header/format block validated;

date format validation only when dashboard dateColumn === true;

one PASS row if clean.

Remove default "General" validation from Section F. The current code is still validating all expected headers against number formats, causing false failures.

Priority 3 — Make text the governed non-date default
The script already has defaultTextFormat: "@", but it is not the active fallback for non-date format validation.

Recommended behavior:

date columns: validate/apply mm/dd/yyyy or dashboard date format;

explicitly numeric columns: validate/apply explicit dashboard number format;

ordinary non-date text columns: apply text format where needed but do not fail Section F because Google Sheets reports a non-identical default display format.

Priority 4 — Build a real Raw Data template-application path
Raw Data currently calls a function that intentionally does not apply template formatting.

For v1.5.48, add a governed function such as:

applyTemplateFormattingToExistingOutputSheet_()

It should:

use Template - Raw Data;

apply template formatting to the existing final Raw Data sheet;

keep only one final formatted sheet;

not leave a source Raw Data sheet and separate formatted copy;

apply widths, header/title/data styles, hidden columns, date/text formats, freeze/filter where governed.

Priority 5 — Remove row-resize work for templates with 1000 rows or less
The script still calls row resizing during every full template build.

The helper avoids deleting extra rows but still does work.

For v1.5.48:

if rowCount <= 1000, do not call row resize at all;

do not record a row-resize timing step for those templates;

only add rows when dashboard requires more than 1000 rows.

This should directly reduce the Create Templates timing bottleneck.

Priority 6 — Rebuild templates/formatting architecture before improving processing
Yes — for templates and formatting, I recommend rebuilding the Create Templates path cleanly, but not rebuilding business processing yet.

The rebuild should be limited to:

Dashboard config loading.

Resolved global/default formatting.

Template creation/refresh.

Template-to-output formatting.

Dashboard Quality Section F.

Timing/report write behavior.

Do not rebuild Demo P, Master List, Care Plan, Banners, Monthly Change business logic yet.
Stabilize QA/Templates/Formatting first, then modularize.

Direct Answer: Is templates/formatting using One Pass Processing?
No.

The current template builder is dashboard-driven but still performs multiple formatting passes and multiple Apps Script range operations: title rows, header row, data rows, widths, number formats, hidden columns, filters, subheaders, and row heights are all applied separately.

For v1.5.48, the better architecture is:

resolve all dashboard/default formatting into one in-memory plan;

write template values and formats in the fewest practical range operations;

validate only governed structural/date requirements in Section F;

apply template formatting to output sheets through one governed template-application function.

git fetch origin --prune
git checkout -B work origin/main
git status -sb
git log -1 --oneline
find . -maxdepth 2 -type f | sort | head -300
Copy
Startup / Sync Verification Complete
I ran the requested commands and did not modify files.

Command Results
✅ git fetch origin --prune

Fetched latest origin/main.

origin/main advanced from 67ce8fd to 6a96240.

✅ git checkout -B work origin/main

Reset local work branch to origin/main.

Branch work is tracking origin/main.

✅ git status -sb

Current status: ## work...origin/main

No working-tree changes shown.

✅ git log -1 --oneline

Current commit: 6a96240 Commit

✅ find . -maxdepth 2 -type f | sort | head -300

Confirmed current key files include:

AGENT.md

README.md

Current_Production/v1.5.47.1_Production_Script.txt

Current_Production/v1.5.47_Production_Script.txt

Report/v1.5.47.1 - Dashboard Quality Report (1).pdf

Report/v1.5.47.1 - Framework Timing Report (1).pdf

Report/v1.5.47.1 - Google Apps Scripts Executions (1).pdf

spec/v1.9_Master_List_Framework_Specifications_Final_Governing_Edition.pdf

spec/v1.4.26_Production_Script_Archive.txt

spec/v1.4.26.1_Production_Script_Archive.txt

src/Code.gs

Repository State
Branch: work

Tracking: origin/main

Current commit: 6a96240

Remote: https://github.com/tabs912/ML.git

Sync status: up to date with origin/main

Working tree: clean

Governance Notes
AGENT.md confirms the framework must maintain Single File Architecture, Dashboard Driven Framework, Dashboard Controlled Templates, Template First Formatting, Primary PMR

Architecture, One Pass Processing, Dashboard Quality Architecture, Timing Framework, and Validation Framework.

README.md confirms the framework remains in Production Stabilization and should not introduce modular architecture without governance approval.

README.md also confirms the expected One Pass Processing model is Read Once → Process In Memory → Write Once.

Missing Artifact Check
No required top-level startup artifacts are missing:

AGENT.md present.

README.md present.

Current_Production/ present.

spec/ present.

Requested (1) report files are now present under Report/.

Readiness status: ready for next instruction.
1. Critical Issues
1.1 — Requested (1) report files are not present in the repo
You asked me to review:

Report/v1.5.47.1 - Dashboard Quality Report(1).pdf

Report/v1.5.47.1 - Framework Timing Report(1).pdf

Report/v1.5.47.1 - Google Apps Scripts Executions(1).pdf

Those exact files are not present after resetting to origin/main. The available report files are:

Report/v1.5.47.1 - Dashboard Quality Report.pdf

Report/v1.5.47.1 - Framework Timing Report.pdf

Report/v1.5.47.1 - Google Apps Scripts Executions.pdf

I reviewed the available v1.5.47.1 reports instead. This matters because if the (1) files are a newer run, the analysis may not reflect the latest workbook state.

1.2 — Version must move to v1.5.48
You are correct: the next corrected script should be v1.5.48, not another reuse of v1.5.47.1.

Current script still has:

const MASTER_LIST_MERGE_ML_VERSION = "1.5.47";
even though the file is named v1.5.47.1_Production_Script.txt.

Critical recommendation: create a new file:

Current_Production/v1.5.48_Production_Script.txt
and set:

const MASTER_LIST_MERGE_ML_VERSION = "1.5.48";
Do not keep patching v1.5.47.1.

1.3 — Raw Data is not actually applying template formatting
You stated: “Raw Data has no template formatting.” The current code confirms why.

formatRawData() calls:

applyDashboardTemplateFormattingToActiveReportSheet_(...)
and logs:

Apply Raw Data dashboard/template formatting in place

But applyDashboardTemplateFormattingToActiveReportSheet_() does not apply real template formatting. It only writes metadata/date cells, locks row heights, clears caches, and returns. The comment explicitly says runtime formatting is restricted and does not reapply dashboard formatting, hidden columns, filters, freezes, widths, or date/number formatting outside the template builder.

So the Raw Data function currently says it applies dashboard/template formatting, but the called function intentionally does not apply the template format.

Critical correction needed in v1.5.48: Raw Data needs a real template-format application path. Options:

copy formatting from Template - Raw Data onto the active Raw Data sheet, or

rebuild Raw Data through a template-copy output pattern, then delete/archive the original source, or

create a dedicated applyTemplateFormattingToExistingSheet_() that copies formats, widths, hidden columns, freeze/filter behavior, and date/text formats from the governed template onto the current sheet.

Given your architecture goal — one final formatted sheet, no raw/source sheet left — option 3 is probably best.

1.4 — Section F is still failing because default number format validation is still active in the checked-in script
The available Dashboard Quality Report shows Section F failures for many columns:

Date/number format mismatch ... expected General but found 0.################
The checked-in v1.5.47.1 script still has:

defaultNumberFormat: "General"
The Section F validator still checks expected number formats for all expected headers, not just date columns. It pushes:

Date/number format mismatch ...
for any mismatch.

This conflicts with your requirement: Section F should only check date format for columns where Date Format / Date Column is true.

Critical correction needed in v1.5.48: Section F should not validate default/general/text number formats. It should only validate date formats for columns marked as date columns.

1.5 — Row resizing for templates with 1000 rows or less is still present in the checked-in script
The timing report shows slow row-resize steps for templates at or below 1000 rows:

Master List — 500 rows

Monthly Change — 1000 rows

Disenrolled Exclusion — 1000 rows

Master List Change Log — 1000 rows

The current build path still calls:

resizeSheetRows_(sheet, rowCount);
for all templates.

Critical correction needed in v1.5.48: do not call row resizing at all when rowCount <= 1000.

2. Performance Bottlenecks

2.1 — Create / Refresh All Templates remains critical

The Framework Timing Report shows:

Create / Refresh All Templates: 268.269 sec

Status: CRITICAL

Benchmark: 240 sec

Variance: +28.269 sec

The Apps Script execution report shows actual wall-clock runtime:

createOrRefreshAllReportTemplates: 303.965 sec

Main contributors:

row resizing for templates at or below 1000 rows,

broad template formatting passes,

date/number formatting on all columns,

template validation/reporting overhead.

The current script still performs row resizing unconditionally in the template build path.

2.2 — Dashboard Quality Validate Templates is the largest current bottleneck

The Framework Timing Report shows:

Dashboard Quality Validate Templates: 159.974 sec

Status: CRITICAL

Benchmark: 30 sec

Variance: +129.974 sec

The Apps Script execution report shows:

runDashboardQualityValidateTemplates: 191.581 sec

The main cause appears to be Section F staging:

Dashboard Quality Section F staged: 126.532 sec

This is likely inflated by the huge repeated Date/number format mismatch text for non-date columns.

Correction path: once Section F stops checking non-date/default number formats, the Section F staged time should drop substantially.

2.3 — Dashboard Quality Start Up improved but is not the current target
Dashboard Quality Start Up is:

25.2 sec

PASS in the Framework Timing Report

38.897 sec in Apps Script wall-clock execution

This is not where I would focus next.

3. Architecture Issues
3.1 — The formatting/template/QA layer has become inconsistent enough to justify a rebuild
You asked: “Should you rebuild the Create Templates from scratch?”

My answer: Yes — rebuild the QA / Templates / Formatting engine from scratch for v1.5.48.

But do not rebuild the processing/business workflow yet.

The current architecture has contradictions:

Raw Data says it applies template formatting, but the function intentionally does not apply template formatting.

Section F is intended to validate structure, but it is failing on non-date number formats.

Template creation still resizes rows unnecessarily for <=1000-row templates.

Report/script versioning is not consistent.

This is enough evidence that incremental patching is now high risk.

3.2 — Separate modules before processing improvements

Your stated goal is correct: focus on QA, Templates, and Formatting first, then split those into modules before improving processing.

Recommended module boundaries for v1.5.48:

Dashboard Config Loader

reads global settings,

sheet definitions,

title rows,

column definitions,

behaviors,

headers.

Template Builder

creates/refreshes templates,

applies global defaults,

applies explicit overrides,

skips row resizing for <=1000 rows,

builds Monthly Change subsection blocks.

Template Formatter

applies template formatting to an existing sheet,

used by Raw Data in-place formatting.

Template QA / Section F

validates structure only,

validates date formats only for date columns,

validates Monthly Change subsection blocks.

Framework Timing / Dashboard Quality Reporter

reports compact findings,

avoids huge repeated error strings.

Processing should stay untouched until these are stable.

3.3 — Section F requirements should be explicitly encoded
You gave the Section F requirements. The validator should be limited to those:

Load dashboard configuration.

For each governed sheet definition, locate its template.

Verify the template exists.

Verify template title rows exist where required.

Verify template header row exists.

Compare template headers to dashboard header definitions.

Identify missing template headers.

Identify unexpected template headers.

Validate required template sections/subsections.

For Monthly Change, verify:

subsection title row,

spacer row,

subsection header row,

formatting block,

copied headers in 8pt font.

Report template structure problems.

If no issues exist, write one PASS summary row.

The current Section F validator still includes non-date number-format validation and should be refactored to match only the above requirements.

4. Prioritized Remediation Plan
Priority 0 — Create v1.5.48 and stop patching v1.5.47.1
Copy the current latest script into:

Current_Production/v1.5.48_Production_Script.txt
Set:

const MASTER_LIST_MERGE_ML_VERSION = "1.5.48";
Keep v1.5.47.1 as historical output.

Priority 1 — Rebuild Section F validator to match your requirements only
Remove from Section F:

default/general number-format validation,

non-date number-format validation,

width validation unless you explicitly want it,

giant repeated mismatch text.

Keep in Section F:

template exists,

title rows exist,

header row exists,

headers match dashboard definitions,

missing headers,

unexpected headers,

Monthly Change subsection block validation,

date-format validation only for date columns.

Priority 2 — Fix Raw Data template formatting
Raw Data must receive real template formatting. The current runtime function does not do that.

Recommended v1.5.48 implementation:

applyTemplateFormattingToExistingSheet_(sheet, "Template - Raw Data")
It should copy from the template:

row 1–4 formatting,

header row formatting,

data row base formatting,

column widths,

hidden column settings,

frozen rows/columns,

filter,

date/text formats,

tab color if needed.

Then Raw Data can still remain a single final formatted sheet.

Priority 3 — Remove row resizing for <=1000-row templates
In the template build path:

```
if (rowCount > 1000) {
  resizeSheetRows_(sheet, rowCount);
} else {
  markFrameworkStep_(timing, "Resize rows skipped - default 1000 rows sufficient...");
}
```
Also make resizeSheetRows_() return immediately for rowCount <= 1000.

This directly targets the current timing bottlenecks for Master List, Monthly Change, Disenrolled Exclusion, and Master List Change Log.

Priority 4 — Rebuild Create Templates engine, but only the template/formatting layer
Yes, I recommend rebuilding Create Templates from scratch for v1.5.48, but with a narrow scope:

Rebuild:

dashboard config read model,

template creation/refresh,

template formatting,

Monthly Change subsection blocks,

Section F QA,

timing around those steps.

Do not rebuild yet:

Process Demo P,

Create Master List,

Update Master List,

Monthly Change processing logic,

Care Plan sync processing,

Banner sync processing.

This keeps the next iteration focused and reduces risk.

Direct Answer: Should Create Templates be rebuilt from scratch?
Yes — rebuild Create Templates from scratch for v1.5.48, but only the QA / Templates / Formatting layer.

The current Create Templates path has accumulated too many conflicting patches:

version mismatch,

Raw Data formatting no-op,

Section F validating the wrong things,

row resizing for sheets that should not resize,

large Dashboard Quality output bottlenecks.

A clean v1.5.48 template engine will be safer than more incremental patching.
Priority 1 — Section F validation scope
Status: Corrected in code.

Section F now focuses on the template-structure requirements you listed:

title row presence,

template header comparison,

missing headers,

unexpected headers,

Monthly Change subsection validation,

date-format validation only for columns marked Date Column = true.

The validator now checks title row 1, compares dashboard headers to template headers, identifies missing headers, and identifies unexpected headers.

The date-format validation now skips every column unless def.dateColumn === true, so Section F should no longer fail default text/non-date columns for General vs 0.###############.

Runtime validation still needed: rerun Dashboard Quality Validate Templates to confirm Section F no longer reports the previous number-format mismatch failures.

Priority 2 — Monthly Change subsection/header block
Status: Corrected in code.

Monthly Change subsection generation now pulls subsection header labels from the current sheet definition’s sheet type instead of the alternate SHEET_TYPE.MONTHLY_CHANGE key. This should fix the blank header row issue shown in your second screenshot.

Monthly Change validation now checks:

subsection title text,

title formatting,

spacer row color,

spacer row height,

spacer row blankness,

header row color,

header row 8pt font size,

header row content matching Monthly Change dashboard headers.

Runtime validation still needed: rebuild the Monthly Change template and visually confirm that each subsection has:

title row,

dark/blank spacer row,

header row below it with copied headers in 8pt font.

Priority 3 — Default format as text, not General number
Status: Corrected in code.

The default non-date format is now plain text (@) instead of General.

This aligns with your request: default format by text, not General number.

Runtime validation still needed: rerun template creation and confirm non-date/default columns no longer show as 0.##################.

Priority 4 — Remove row resizing for templates with 1000 rows or less
Status: Corrected in code.

The template build path now skips row resizing when rowCount <= 1000 and logs that the Google Sheets default 1000 rows are sufficient.

The row-resize helper also returns immediately when rowCount <= 1000, so it avoids even checking getMaxRows() for those templates.

Runtime validation still needed: rerun Create / Refresh All Templates and confirm these slow timing entries disappear:

Resize rows: Template - Master List (500 rows)

Resize rows: Template - Monthly Change (1000 rows)

Resize rows: Template - Disenrolled Exclusion (1000 rows)

Resize rows: Template - Master List Change Log (1000 rows)

Remaining Required Validation
The code corrections are in place, but the next Apps Script run should confirm:

Create / Refresh All Templates no longer spends time resizing rows for templates with 1000 rows or fewer.

Dashboard Quality Validate Templates no longer fails Section F for non-date/default number formats.

Monthly Change subsection header rows now populate correctly.

Section F only reports true template-structure issues and date-column format issues.